

ECE276A Project 2 Report: LiDAR-Based SLAM

Stephen Huang

Department of Electrical and Computer Engineering
University of California, San Diego

La Jolla, U.S.A.
sth004@ucsd.edu

Abstract—This project report presents a LiDAR-based simultaneous localization and mapping (SLAM) system that integrates wheel encoder, inertial measurement unit (IMU), and Hokuyo UTM-30LX LiDAR measurements. Because these sensor measurements are subject to noise and drift that accumulate over time, accurate pose estimation requires multi-stage optimization. We first apply the Iterative Closest Point (ICP) algorithm to refine relative pose estimates between scans. Then, a factor graph optimization is performed to improve global pose consistency and reduce accumulated error. Finally, we incorporate RGBD data to construct a texture map from the optimized poses. Together, these components improves localization accuracy and map quality in the presence of sensor drift.

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) estimates a robot trajectory while building a map of an unknown environment from sensor measurements collected over time. In many robotics settings, however, reliable localization and mapping is difficult due to unreliable sensors which accumulate drift. These effects degrade scan alignment and can distort the resulting map, impacting tasks such as planning and navigation.

To address these limitations, we use encoder and IMU measurements to obtain an initial differential-drive pose estimate, and then refine the trajectory using 2D LiDAR scan matching and pose graph optimization. The estimated trajectory is used to construct a 2D occupancy grid map of the environment, and RGBD measurements are projected onto the same plane to build a floor texture map. The end goal is to reduce the effect of cumulative drift and improve the global consistency and visual quality of the resulting maps.

Our approach estimates 2D robot poses in $SE(2)$ and refines them in multiple stages. We first compute an odometry trajectory from encoder and IMU data, then improve relative pose estimates between consecutive LiDAR scans using ICP. We then optimize the full trajectory with a pose factor graph and loop-closure constraints, and finally use the optimized poses for geometric and texture mapping. Our full process consists of:

- a differential-drive motion estimate formed from encoder linear motion and IMU yaw-rate measurements,
- a LiDAR scan-matching stage formed by ICP aligned with Kabsch algorithm,
- a pose graph optimization stage with loop-closure factors to reduce accumulated drift,

- an occupancy mapping stage using Bresenham ray tracing and log-odds updates, and
- a texture mapping stage formed by projecting RGBD floor points into the 2D map frame.

This produces a LiDAR-based SLAM based on odometry and optimized by scan matching and factor graph optimization, with an RGBD texture map for visualization.

II. PROBLEM FORMULATION

Consider a differential-drive robot navigating in an unknown, 2D indoor environment. We want to estimate the robot trajectory and construct a map of the environment from asynchronous sensor measurements. In addition we want to assign floor-color information from RGBD data to the same 2D map.

A. State and Pose Representation

Let the robot state at time t be

$$\mathbf{x}_t = [x_t \ y_t \ \theta_t]^\top \in \mathbb{R}^2 \times (-\pi, \pi], \quad (1)$$

with associated pose

$$T_t := \begin{bmatrix} R(\theta_t) & \mathbf{p}_t \\ \mathbf{0}^\top & 1 \end{bmatrix} = \begin{bmatrix} \cos \theta_t & -\sin \theta_t & x_t \\ \sin \theta_t & \cos \theta_t & y_t \\ 0 & 0 & 1 \end{bmatrix} \in SE(2), \quad (2)$$

where $\mathbf{p}_t = [x_t, y_t]^\top$ and $R(\theta_t) \in SO(2)$. Over a period of time T , we define the unknown state trajectory

$$\mathbf{x}_{0:T} := \{\mathbf{x}_t\}_{t=0}^T. \quad (3)$$

B. Measurements and Timestamp Association

The sensors provide asynchronous timestamped measurements:

$$Z^{\text{enc}} := \{(\tau_i^{\text{enc}}, z_i^{\text{enc}})\}_{i=1}^{N_{\text{enc}}}, \quad (4)$$

$$Z^{\text{imu}} := \{(\tau_i^{\text{imu}}, z_i^{\text{imu}})\}_{i=1}^{N_{\text{imu}}}, \quad (5)$$

$$Z^{\text{lidar}} := \{(\tau_i^{\text{lidar}}, z_i^{\text{lidar}})\}_{i=1}^{N_{\text{lidar}}}, \quad (6)$$

$$Z^{\text{rgb}} := \{(\tau_i^{\text{rgb}}, I_i)\}_{i=1}^{N_{\text{rgb}}}, \quad (7)$$

$$Z^{\text{d}} := \{(\tau_i^{\text{d}}, D_i)\}_{i=1}^{N_{\text{d}}}, \quad (8)$$

where z_i^{enc} denotes encoder counts, z_i^{imu} is IMU data, and $z_i^{\text{lidar}} \in \mathbb{R}^{N_\ell}$ is the LiDAR range vector for scan z_i^{lidar} , with $N_\ell = 1081$ beams.

Each measurement is associated with a Unix timestamp. We must align measurements in time (or to the nearest pose timestamp) when constructing trajectory and map estimates.

C. Motion Model

Let the odometry control input at time t be

$$\mathbf{u}_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix}, \quad (9)$$

where v_t is linear velocity computed from encoder data and ω_t is yaw-rate from the IMU.

We write the motion model as

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t) + \mathbf{v}_t, \quad (10)$$

where $\mathbf{v}_t$ captures odometry uncertainty (e.g., wheel slip, quantization, IMU bias/drift).

D. Occupancy Grid Mapping

Let $g(\tau)$ be the nearest-past association function that maps a measurement timestamp to a pose. Then, $T_{g(\tau_i^{\text{lidar}})}$ is the pose associated with scan i . Let ${}_rT_\ell \in SE(2)$ be the extrinsic matrix from LiDAR to robot body frame. We define the cartesian world coordinates for a LiDAR scan as

$$\mathbf{x}_{g(\tau_i^{\text{lidar}})} := \phi(T_{g(\tau_i^{\text{lidar}})}, {}_rT_\ell, z_i^{\text{lidar}}) \quad (11)$$

Let $\mathbf{m}$ denote the occupancy grid map. The LiDAR observation model takes the form

$$z_i^{\text{lidar}} = h(\mathbf{x}_{g(\tau_i^{\text{lidar}})}, \mathbf{m}) + \mathbf{n}_i, \quad (12)$$

where $\mathbf{n}_i$ denotes measurement noise. Given a trajectory $\mathbf{x}_{0:T}$, the LiDAR-only mapping objective can be written as

$$\min_{\mathbf{m}} \sum_{i=1}^{N_{\text{lidar}}} \left\| z_i^{\text{lidar}} - h(\mathbf{x}_{g(\tau_i^{\text{lidar}})}, \mathbf{m}) \right\|_2^2. \quad (13)$$

E. SLAM Problem

Given the initial state $\mathbf{x}_0$, odometry controls $\mathbf{u}_{0:T-1}$, and asynchronous LiDAR measurements $Z^{\text{lidar}} = \{(\tau_i^{\text{lidar}}, z_i^{\text{lidar}})\}_{i=1}^{N_{\text{lidar}}}$ with observation model $h(\cdot)$ and motion model $f(\cdot)$, the SLAM problem jointly estimates the state trajectory $\mathbf{x}_{1:T}$ and the occupancy grid map $\mathbf{m}$ by

$$\min_{\mathbf{x}_{1:T}, \mathbf{m}} \sum_{i=1}^{N_{\text{lidar}}} \left\| z_i^{\text{lidar}} - h(\mathbf{x}_{g(\tau_i^{\text{lidar}})}, \mathbf{m}) \right\|_2^2 + \sum_{t=0}^{T-1} \left\| \mathbf{x}_{t+1} - f(\mathbf{x}_t, \mathbf{u}_t) \right\|_2^2, \quad (14)$$

where $g(\tau_i^{\text{lidar}})$ is the nearest-past timestamp association.

F. Texture Map Problem

In addition to mapping, we want a 2D floor texture map that assigns color to the 2D grid. Let the texture map be

$$\mathbf{C} \in \mathbb{R}^{W_m \times H_m \times 3}, \quad \mathbf{c}_{i,j} \in \mathbb{R}^3, \quad (15)$$

where $\mathbf{c}_{i,j}$ stores an RGB value for cell (i, j) . The RGB and disparity (depth) streams are

$$Z^{\text{rgb}} := \{(\tau_i^{\text{rgb}}, I_i)\}_{i=1}^{N_{\text{rgb}}}, \quad (16)$$

$$Z^d := \{(\tau_i^d, D_i)\}_{i=1}^{N_d}. \quad (17)$$

Using timestamp association, we want to project RGBD measurements onto the floor plane, producing colored observations for each grid cell.

Given the trajectory $\mathbf{x}_{0:T}$ and RGBD measurements (Z^{rgb}, Z^d) , the texture mapping problem is to estimate $\mathbf{c}$ such that its cell colors are consistent with the observed floor colors.

III. TECHNICAL APPROACH

A. Differential Drive Odometry Estimation

We begin by estimating the robot's initial 2D trajectory using wheel encoder and IMU measurements under a differential-drive kinematic model. Since the encoder and IMU streams are sampled asynchronously, we align encoder to IMU timestamps using a nearest-past association.

1) *Preprocessing and Time Alignment:* For each

$$Z_i^{\text{enc}} := (\tau_i^{\text{enc}}, z_i^{\text{enc}}), \quad z_i = [FR, FL, RR, RL],$$

where first letter F = front, R = rear and second letter L = left, R = right. We are given the constant

$$\text{METERS_PER_TIC} = 0.0022,$$

We can then compute the distance traveled per time step for the left and right wheels:

$$\Delta m_{R,i} = \frac{FR_i + RR_i}{2} \cdot 0.0022, \quad \Delta m_{L,i} = \frac{FL_i + RL_i}{2} \cdot 0.0022. \quad (18)$$

With encoder time increments $\Delta \tau_i^{\text{enc}}$, the corresponding wheel-side velocities are

$$v_{R,i} = \frac{\Delta m_{R,i}}{\Delta \tau_i^{\text{enc}}}, \quad v_{L,i} = \frac{\Delta m_{L,i}}{\Delta \tau_i^{\text{enc}}}, \quad (19)$$

and the estimated linear velocity is

$$v_i = \frac{v_{L,i} + v_{R,i}}{2}. \quad (20)$$

From the IMU, we use the yaw-rate measurement $\omega(t)$ (the z -axis angular velocity). Let IMU timestamps be $\{\tau_k^{\text{imu}}\}$ and encoder timestamps by $\{\tau_i^{\text{enc}}\}$, we align the linear velocity v_i to each IMU timestamp τ_k^{imu} via the nearest-past association function

$$\phi(\{\tau_i\}_{i=0}^{N-1}, \tau) := \max\{i \in \{0, \dots, N-1\} : \tau_i \leq \tau\}. \quad (21)$$

As there are more IMU timestamps, we use $\phi(\{\tau_i^{\text{enc}}\}, \{\tau_k^{\text{imu}}\})$ to align the timestamps.

2) *Pose Estimation:* We represent the 2D pose as $T_k \in SE(2)$ and integrated using the body frame twist formed from (v_k, ω_k) :

$$\xi_k = \begin{bmatrix} 0 & -\omega_k & v_k \\ \omega_k & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (22)$$

Given IMU timestep $\Delta \tau_k$, we update the pose by performing the discrete time update step for differential drive:

$$T_{k+1} = T_k \exp(\Delta \tau_k \xi_k), \quad (23)$$

where $\exp(\cdot)$ denotes the matrix exponential. We initialize with $T_0 = I_3$ and perform the pose estimation over all k LiDAR data points. If an invalid timestep is encountered ($\Delta t_k \leq 0$), we skip the update and retain the previous pose.

B. LiDAR Scan Projection to Cartesian Coordinates

The Hokuyo UTM-30LX provides a 2D scan consisting of $N_\ell = 1081$ range measurements over a fixed angle field of view. The sensor model includes scalar parameters $\alpha_{\min}$, $\alpha_{\max}$, $\Delta\alpha$, $r_{\min}$, and $r_{\max}$, corresponding to the valid angle limits, angular increment, and valid range limits, respectively. For each scan captured at timestamp τ , the LiDAR reports a range vector

$$\mathbf{z}^{\text{lidar}}(\tau) = [r_0(\tau), r_1(\tau), \dots, r_{N_\ell-1}(\tau)]^\top \in \mathbb{R}^{N_\ell}. \quad (24)$$

The i th beam angle is

$$\alpha_i = \alpha_{\min} + i \Delta\alpha, \quad i = 0, 1, \dots, N_\ell - 1. \quad (25)$$

We discard invalid returns by thresholding ranges:

$$V(\tau) := \{i : r_{\min} + \epsilon < r_i(\tau) < r_{\max} + \epsilon\}. \quad (26)$$

where $\epsilon = 0.5$ for our case. For each valid range $r_i(\tau)$ with $i \in V(\tau)$, the corresponding Cartesian point in the LiDAR frame is computed via polar-to-Cartesian conversion:

$$\mathbf{a}_i(\tau) := \begin{bmatrix} x_i(\tau) \\ y_i(\tau) \end{bmatrix} = \begin{bmatrix} r_i(\tau) \cos \alpha_i \\ r_i(\tau) \sin \alpha_i \end{bmatrix} \in \mathbb{R}^2. \quad (27)$$

To express points in the robot body frame, we apply the LiDAR-to-body $\delta_x = 0.29833\text{m}$ translation along the body x-axis. Then, the body frame point is

$$\mathbf{a}_i^{(b)}(\tau) = \mathbf{a}_i^{(\ell)}(\tau) + \begin{bmatrix} \delta_x \\ 0 \end{bmatrix}. \quad (28)$$

Collecting all valid points yields the body frame point cloud

$$\mathbf{m}(\tau) = \left\{ \mathbf{a}_i^{(b)}(\tau) \right\}_{i \in V(\tau)} \in \mathbb{R}^{2 \times N_\ell}, \quad (29)$$

which is used as the input for scan matching and occupancy mapping.

C. ICP Scan Matching

To refine the odometry trajectory, we perform alignment between consecutive LiDAR scans. Given scans $\mathbf{z}_t$, $\mathbf{z}_{t+1}$ at times t and $t+1$, we estimate the relative transform ${}_tT_{t+1}$ that aligns scan $t+1$ to scan t .

1) *Initialization*: For each consecutive scan pair, the initial relative pose is computed from the odometry trajectory:

$${}_tT_{t+1}^{(0)} = (T_t^{\text{odom}})^{-1} T_{t+1}^{\text{odom}}. \quad (30)$$

This serves as the starting pose for ICP.

Furthermore, a KD-tree is built on points from scan $\mathbf{z}_t$ in the body frame for correspondence and error measurement.

2) *2D Kabsch Algorithm*: Given two point clouds $\{(\mathbf{z}_i, \mathbf{m}_i)\}_{i=1}^N$, we define the centroids

$$\bar{\mathbf{z}} = \frac{1}{N} \sum_{i=1}^N \mathbf{z}_i, \quad \bar{\mathbf{m}} = \frac{1}{N} \sum_{i=1}^N \mathbf{m}_i. \quad (31)$$

and centered points

$$\delta \mathbf{z}_i = \mathbf{z}_i - \bar{\mathbf{z}}, \quad \delta \mathbf{m}_i = \mathbf{m}_i - \bar{\mathbf{m}}. \quad (32)$$

The Q matrix is

$$Q = \sum_{i=1}^N \delta \mathbf{m}_i \delta \mathbf{z}_i^\top. \quad (33)$$

Let the singular value decomposition of Q be

$$Q = U \Sigma V^\top. \quad (34)$$

Then, the solution is

$$R = U \text{diag}(1, \det(UV^\top)) V^\top, \quad (35)$$

The 3D Kabsch Algorithm uses a $\text{diag}(1, 1, \det(UV^\top))$ matrix. The corresponding translation is

$$\mathbf{p} = \bar{\mathbf{m}} - R\bar{\mathbf{z}}. \quad (36)$$

3) *ICP Algorithm*: We implement the ICP algorithm as follows:

Algorithm 1 ICP Optimization with Kabsch Updates

```

1: function ICPOPTIMIZE( $P, M, R_0, \mathbf{p}_0, K_{\max}$ )
2:    $R \leftarrow R_0, \mathbf{p} \leftarrow \mathbf{p}_0$ 
3:    $\tilde{P} \leftarrow \{R\mathbf{z} + \mathbf{p} : \mathbf{z} \in P\}$ 
4:    $\epsilon_{\text{best}} \leftarrow +\infty, R_{\text{best}} \leftarrow R, \mathbf{p}_{\text{best}} \leftarrow \mathbf{p}$ 
5:   for  $k \leftarrow 1$  to  $K_{\max}$  do
6:      $(C, I) \leftarrow \text{NEARESTNEIGHBOR}(\tilde{P}, M) \quad \triangleright C =$ 
       correspondences,  $I = \text{inlier}$ 
7:      $\epsilon \leftarrow \text{MSE}(\tilde{P}, C, I)$ 
8:     if  $\epsilon < \epsilon_{\text{best}}$  then
9:        $\epsilon_{\text{best}} \leftarrow \epsilon, R_{\text{best}} \leftarrow R, \mathbf{p}_{\text{best}} \leftarrow \mathbf{p}$ 
10:    end if
11:     $(\Delta R, \Delta \mathbf{p}) \leftarrow \text{KABSCH}(\tilde{P}, C, I)$ 
12:     $\tilde{P} \leftarrow \{\Delta R\mathbf{z} + \Delta \mathbf{p} : \mathbf{z} \in \tilde{P}\}$ 
13:     $R \leftarrow \Delta R R$ 
14:     $\mathbf{p} \leftarrow \Delta R \mathbf{p} + \Delta \mathbf{p}$ 
15:  end for
16:  return  $(R_{\text{best}}, \mathbf{p}_{\text{best}})$ 
17: end function

```

This algorithm gives us the refined poses $\{T_k^{\text{refined}}\}$

D. Factor Graph Optimization

After scan matching, the refined poses are used to construct a pose factor graph in GTSAM. Each robot pose is represented as a graph vertex, and relative poses are represented as factors.

1) *Graph Construction*: We begin by defining a graph a graph

$$G = (V, E)$$

We define the set of vertices as

$$V = \{v_i\}_{i=0}^{N-1}, \quad v_i \in SE(2).$$

and the edges

$$E = \{(i, j)\} \subseteq \{0, \dots, N-1\} \times \{0, \dots, N-1\}. \quad (37)$$

Each edge $(i, j) \in E$ is associated with a relative pose ${}_i T_j$ and a noise model Σ_{ij} .

We begin by adding the first vertex v_0 with prior noise. We first define a diagonal noise model

$$\Sigma_0 = \text{diag}(\sigma_{0,x}^2, \sigma_{0,y}^2, \sigma_{0,\theta}^2)$$

and then apply the noise to v_0

$$v_0 \sim \mathcal{N}(v_0, \Sigma_0),$$

This is implemented in GTSAM as a `PriorFactorPose2`.

Between-factors are added between $(i, i+1)$ using relative transforms computed from the ICP refined poses:

$${}_i T_{i+1} = T_i^{-1} T_{i+1} \quad (38)$$

We also define a diagonal noise model

$$\Sigma_{\text{odom}} = \text{diag}(\sigma_{\text{odom},x}^2, \sigma_{\text{odom},y}^2, \sigma_{\text{odom},\theta}^2)$$

where

$$\sigma_{\text{odom},x}^2 > \sigma_{0,x}^2, \quad \sigma_{\text{odom},y}^2 > \sigma_{0,y}^2, \quad \sigma_{\text{odom},\theta}^2 > \sigma_{0,\theta}^2$$

where ϵ_{odom} is a diagonal noise model with standard deviations. This initializes the odometry factors in between each pose. In GTSAM this is implemented as `BetweenFactorPose2` ($v_i, v_{i+1}, {}_i T_{i+1}, \Sigma_{\text{odom}}$).

Finally, we provide initial estimates for all pose variables $I = \{v_i\}_{i=1}^{N_\ell}$ from ICP trajectory to initialize nonlinear optimization.

2) *Loop-Closure Factors*: Loop-closure factors are added following the given strategies:

- **Fixed-interval loop closure**: candidate pairs are formed at fixed intervals (10 intervals in the implementation).
- **Proximity-based loop closure**: candidate pairs are selected using Euclidian distance with KD-tree over $\mathbf{p} = (x, y)$ positions. The implementation used $\|\Delta \mathbf{p}\|_2 < 0.1$. Furthermore, temporal separation was used to ensure loop-closure factors did not occur when the robot was making turns.
- **Consistency check**: a loop-closure factor is added only if ICP between the candidate scans yields an error below a threshold, 0.001 in the implementation.

For accepted loop closures, a diagonal noise model is added:

$$\Sigma_{\text{loop}} = \text{diag}(\sigma_{\text{loop},x}^2, \sigma_{\text{loop},y}^2, \sigma_{\text{loop},\theta}^2)$$

where

$$\sigma_{\text{loop},x}^2 > \sigma_{\text{odom},x}^2, \quad \sigma_{\text{loop},y}^2 > \sigma_{\text{odom},y}^2, \quad \sigma_{\text{loop},\theta}^2 > \sigma_{\text{odom},\theta}^2$$

the ICP-estimated relative transform is added as a `BetweenFactorPose2`.

3) *Levenberg-Marquardt Optimization*: Since the pose factor graph is a nonlinear least-squares problem (log map), we optimize using Levenberg-Marquardt. After the factor graph is built, we call

```
params = gtsam.LevenbergMarquardtParams()
```

and then pass it into

```
gtsam.LevenbergMarquardtOptimizer(G, I, params)
```

The graph is then optimized by calling `optimize` and the optimized poses are extracted.

E. Occupancy Grid Mapping with Bresenham Ray Tracing

We construct a 2D occupancy grid map (OGM) using the LiDAR scans and the estimated poses. The map is discretized with fixed resolution and bounds, and each cell stores a log-odds occupancy value.

1) *Ray Generation*: For each scan and pose:

- 1) the LiDAR origin is transformed into the world frame,
- 2) each valid LiDAR endpoint is transformed from body frame to world frame,
- 3) world coordinates are converted to grid indices, and
- 4) `bresenham2D` is used to trace each beam from sensor origin to endpoint.

For each beam, all intermediate cells are labeled free, and the terminal cell is labeled occupied.

2) *Log-Odds Update*: We update the occupancy grid in log-odds form with fixed increments:

$$\Delta \ell_{\text{free}} = -1, \quad (39)$$

$$\Delta \ell_{\text{occ}} = +4. \quad (40)$$

Thus, free observations reduce occupancy belief slightly, while occupied observations strongly increase occupancy belief. To prevent unbounded growth, the map values are clipped to

$$\ell \in [-100, 100]. \quad (41)$$

In implementation, these updates are applied with `(np.add.at)` over the free and occupied cell sets returned by the Bresenham routine.

We generate:

- an OGM from ICP refined poses, and
- a second OGM rebuilt from graph optimized poses.

F. RGBD Floor Texture Projection

In addition to occupancy mapping, we construct a 2D floor texture map aligned to the OGM.

1) *Timestamp Association*: Since RGB and disparity frames may be irregular and dropped, each RGB frame is associated with:

- the closest past disparity frame, and
- the closest past robot pose (from the LiDAR/optimized pose timeline).

To do this, we use the $\phi(\cdot)$ function as defined in eq. (21)

2) *Disparity-to-Depth and 3D Back Projection:* Disparity values are converted to depth using the provided calibration:

$$dd = -0.00304d + 3.31, \quad (42)$$

$$\text{depth} = \frac{1.03}{dd}. \quad (43)$$

Let K be the camera's intrinsic matrix. We perform back projection through the following equation:

$$\begin{bmatrix} X_o \\ Y_o \\ Z_o \end{bmatrix} = Z_0 K^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

where K^{-1} is the inverse intrinsic matrix, u is the horizontal pixel index, and v is the vertical pixel index.

We then map disparity pixels to RGB pixels with the following calibration

$$\text{RGB}_u = \frac{526.37u + 19276 - 7877.07dd}{585.051}$$

$$\text{RGB}_v = \frac{526.37v + 16662}{585.051}$$

we also apply a valid mask as follows

$$V = (\text{depth} > 0.1) \wedge (\text{depth} < 30).$$

3) *Camera to World Transformation and Floor Selection:*

The 3-D points are transformed from camera frame to robot body frame using two transformations. Given ϕ, θ, ψ corresponding to yaw, pitch, roll, the regular to optical rotation matrix, and relative position of the camera $\mathbf{p}_o \in \mathbb{R}^3$

$${}_oR_r = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix}$$

we form the extrinsic rotation matrix from optical to robot as follows:

$$R_{\text{ext}} = R_z(\psi)R_y(\theta)R_x(\phi){}_oR_r^{-1}$$

and then finally form the pose

$${}_rT_o = \begin{bmatrix} R_{\text{ext}} & \mathbf{p}_o \\ \mathbf{0}^\top & 1 \end{bmatrix}$$

Then, with a given ${}_wT_r$ pose matrix, we can perform the following frame transformation:

$$\begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = {}_wT_r {}_rT_o \begin{bmatrix} X_o \\ Y_o \\ Z_o \\ 1 \end{bmatrix}$$

Then, we select points that are below a floor threshold τ

$$|Z_w| < \tau_{\text{floor}}, \quad (44)$$

4) *Projection onto the Grid:* The valid floor points are then into OGM grid coordinates, and their RGB values are accumulated per cell. A count map tracks the number of observations per cell. The final texture map is obtained by averaging accumulated colors over observed cells:

$$\mathbf{c}_j = \frac{1}{n_j} \sum_{k=1}^{n_j} \mathbf{c}_j^{(k)}, \quad n_j > 0. \quad (45)$$

This produces a 2D floor texture map that is aligned with the occupancy grid map.

IV. RESULTS

A. Rendered Results for Dataset 20

1) 33%:

a) *OGM:* The following images are OGM:

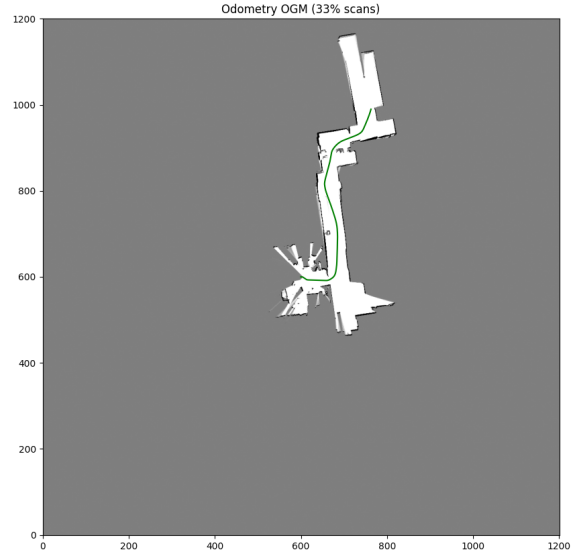


Fig. 1. Dataset 20: Odometry OGM (33%).

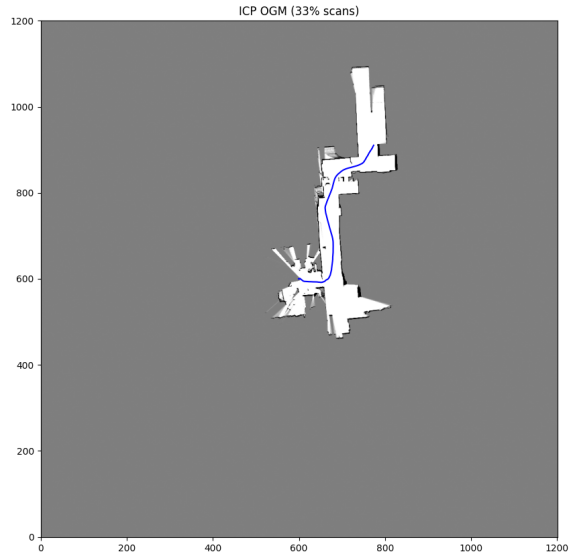


Fig. 2. Dataset 20: ICP refined OGM (33%).

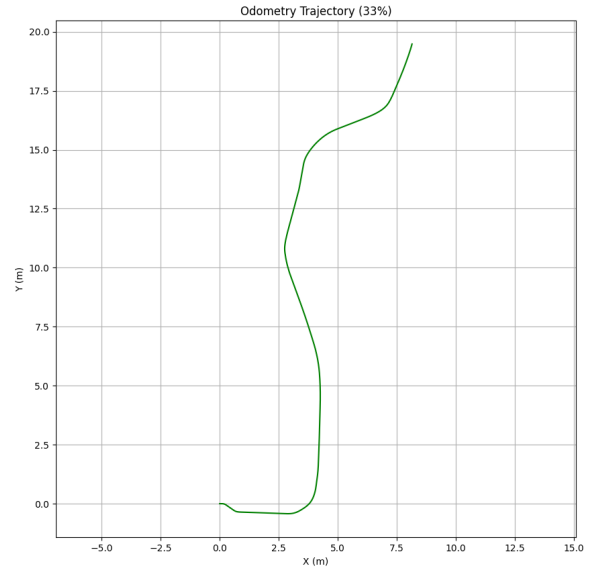


Fig. 4. Dataset 20: Odometry trajectory (33%).

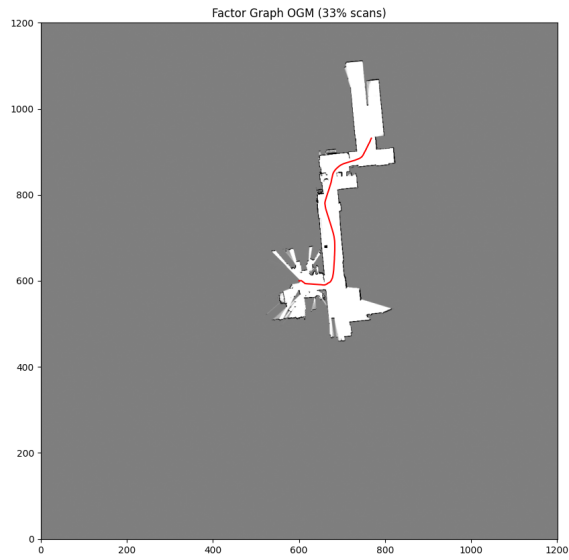


Fig. 3. Dataset 20: Factor-graph optimized OGM (33%).

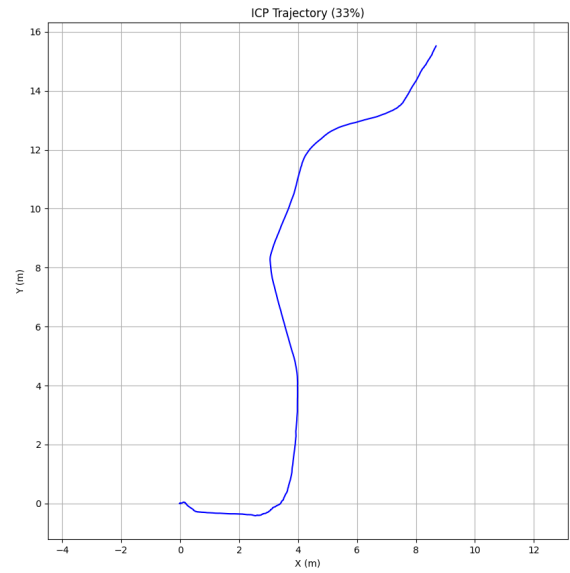


Fig. 5. Dataset 20: ICP refined trajectory (33%).

b) Trajectories: The following images are the trajectories:

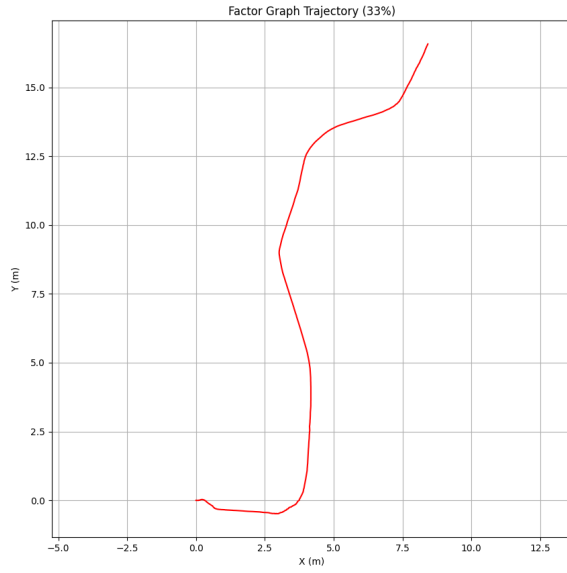


Fig. 6. Dataset 20: Factor-graph optimized trajectory (33%).

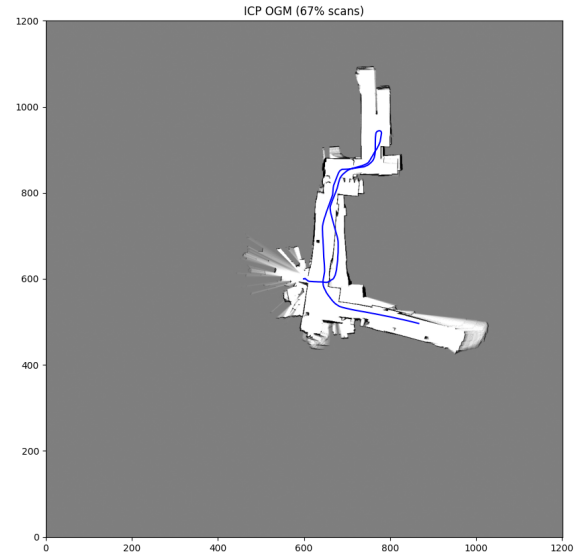


Fig. 8. Dataset 20: ICP refined OGM (67%).

2) 67%:

a) *OGM*: The following images are OGM:

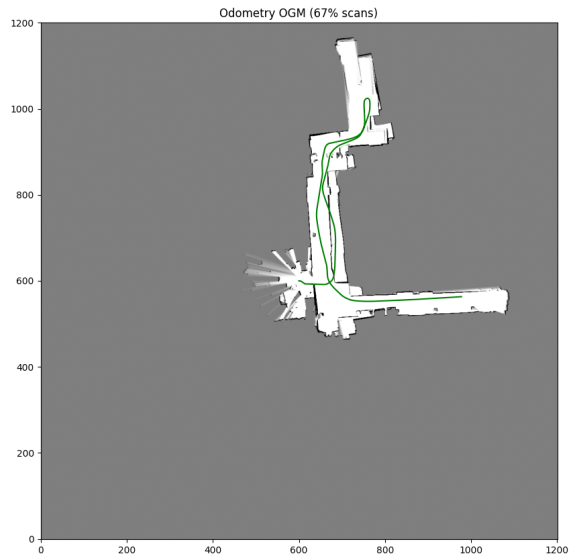


Fig. 7. Dataset 20: Odometry OGM (67%).

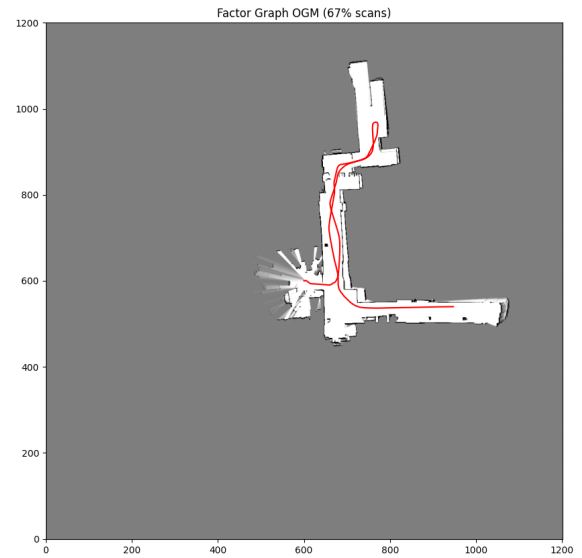


Fig. 9. Dataset 20: Factor-graph optimized OGM (67%).

b) *Trajectories*: The following images are the trajectories:

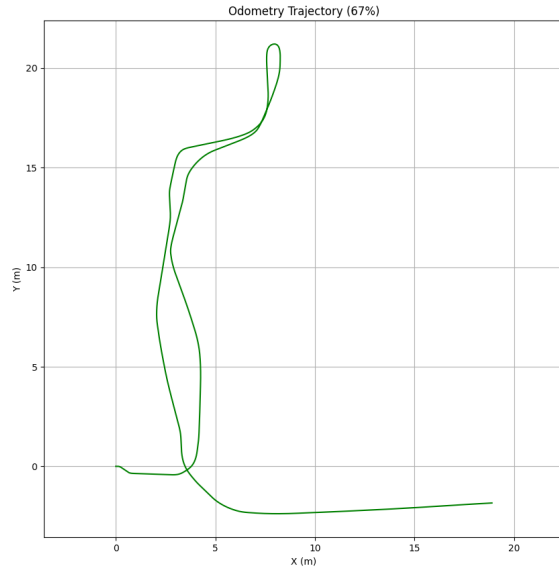


Fig. 10. Dataset 20: Odometry trajectory (67%).

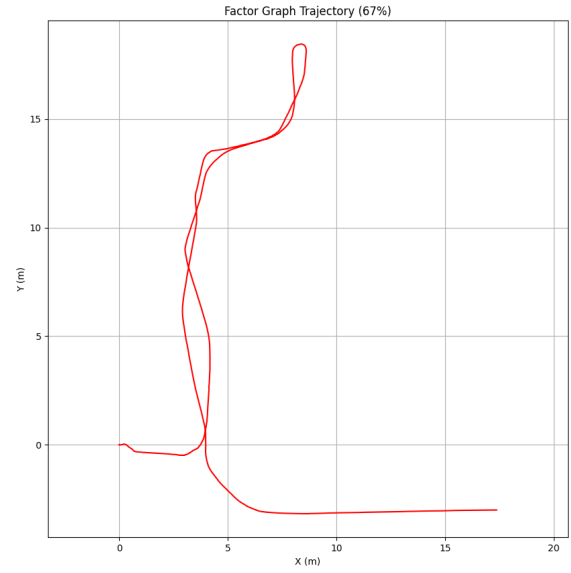


Fig. 12. Dataset 20: Factor-graph optimized trajectory (67%).

3) 100%:

a) OGM: The following images are OGM:

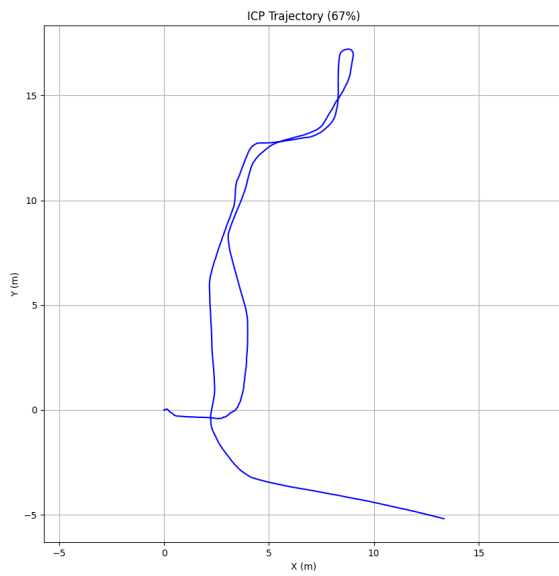


Fig. 11. Dataset 20: ICP refined trajectory (67%).

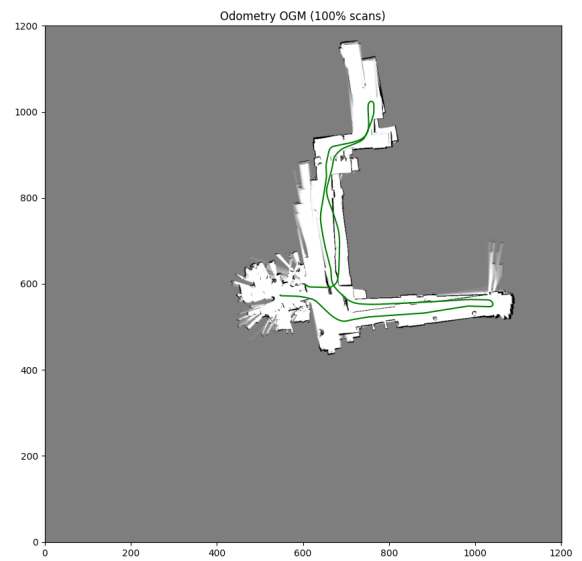


Fig. 13. Dataset 20: Odometry OGM (100%).

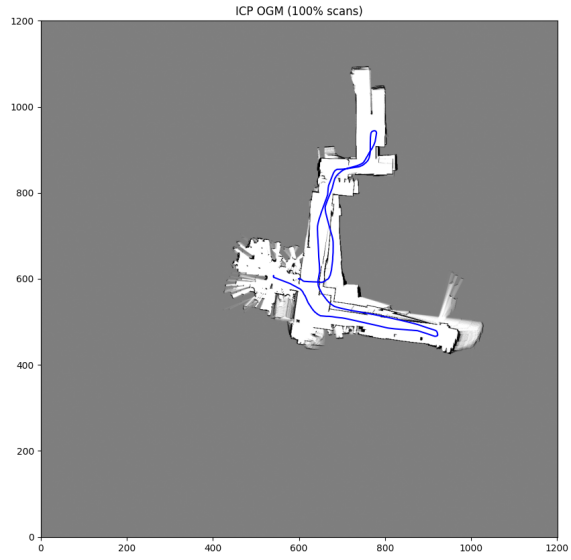


Fig. 14. Dataset 20: ICP refined OGM (100%).

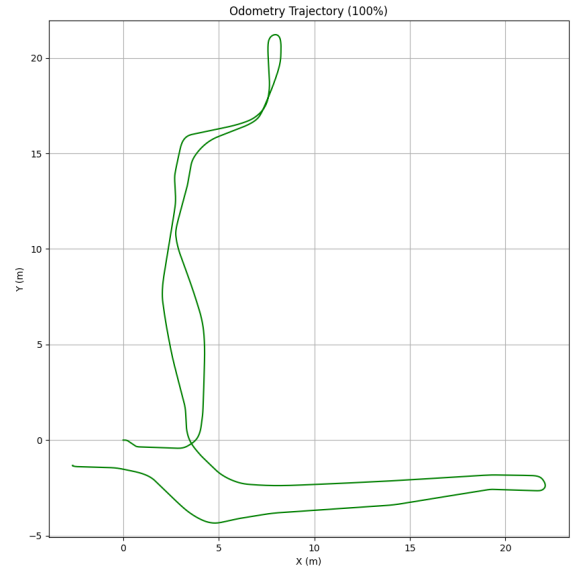


Fig. 16. Dataset 20: Odometry trajectory (100%).



Fig. 15. Dataset 20: Factor-graph optimized OGM (100%).

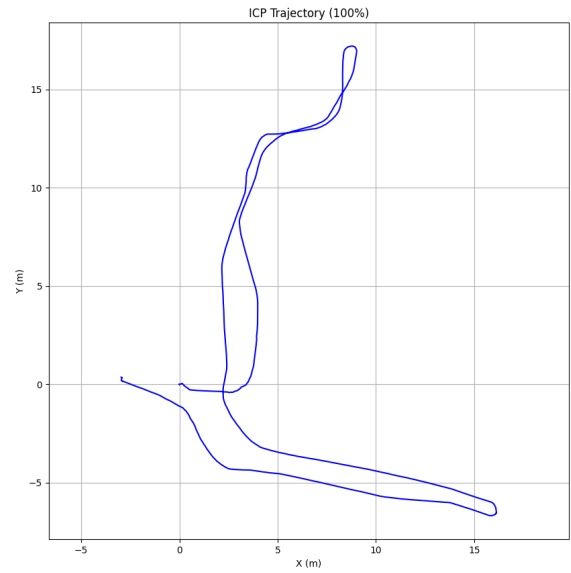


Fig. 17. Dataset 20: ICP refined trajectory (100%).

b) Trajectories: The following images are the trajectories:

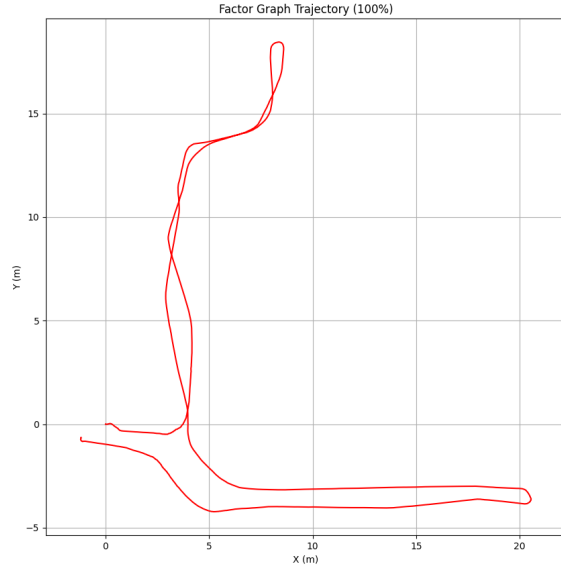


Fig. 18. Dataset 20: Factor-graph optimized trajectory (100%).

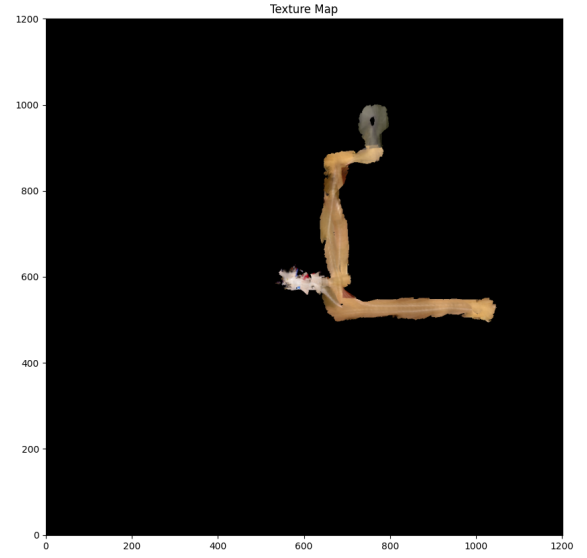


Fig. 20. Dataset 20: Final floor texture map.

B. Rendered Results for Dataset 21

1) 33%:

a) *OGM*: The following images are OGM:

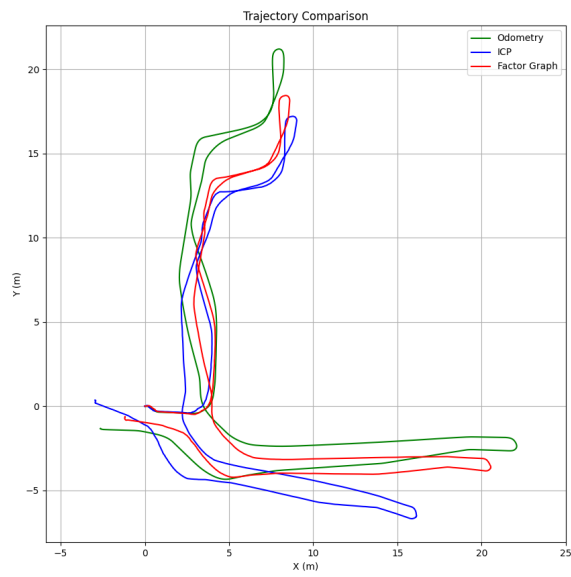


Fig. 19. Dataset 20: Trajectory comparison (odometry vs. ICP vs. factor graph).

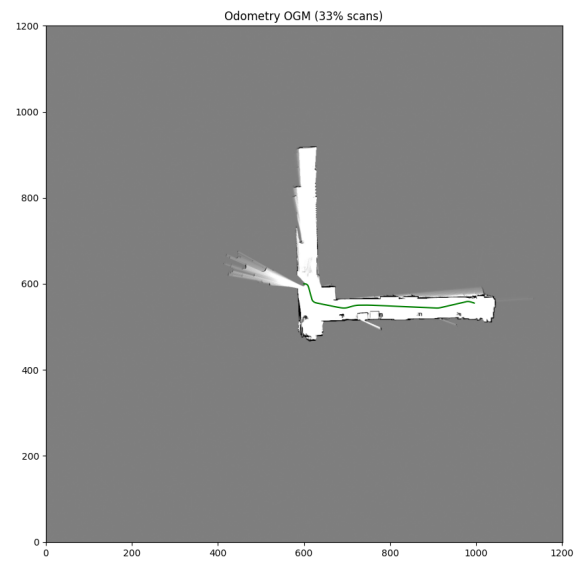


Fig. 21. Dataset 21: Odometry OGM (33%).

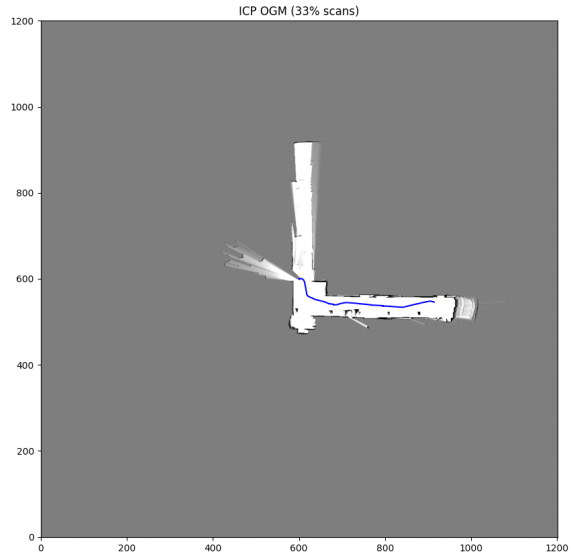


Fig. 22. Dataset 21: ICP refined OGM (33%).

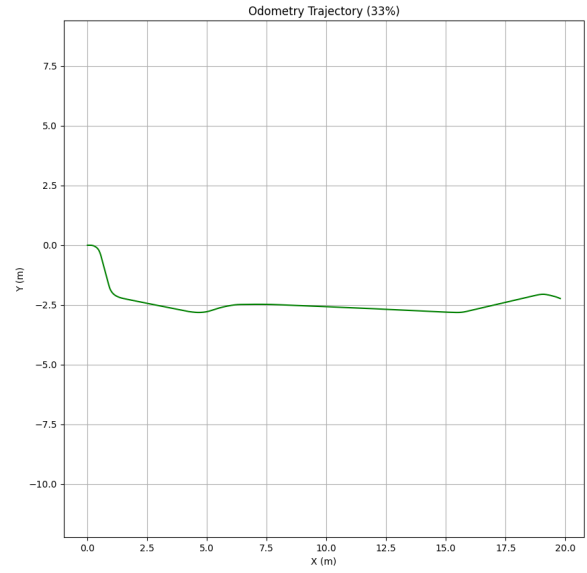


Fig. 24. Dataset 21: Odometry trajectory (33%).

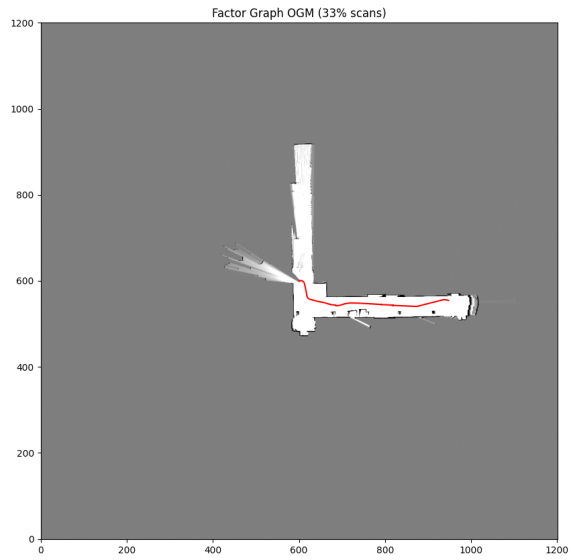


Fig. 23. Dataset 21: Factor-graph optimized OGM (33%).

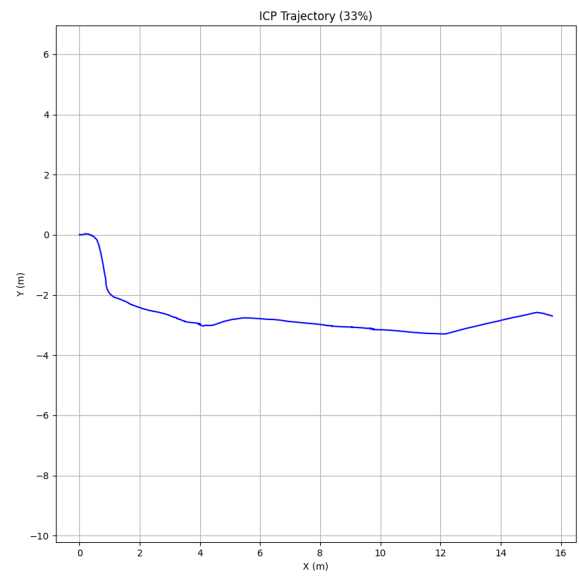


Fig. 25. Dataset 21: ICP refined trajectory (33%).

b) Trajectories: The following images are the trajectories:

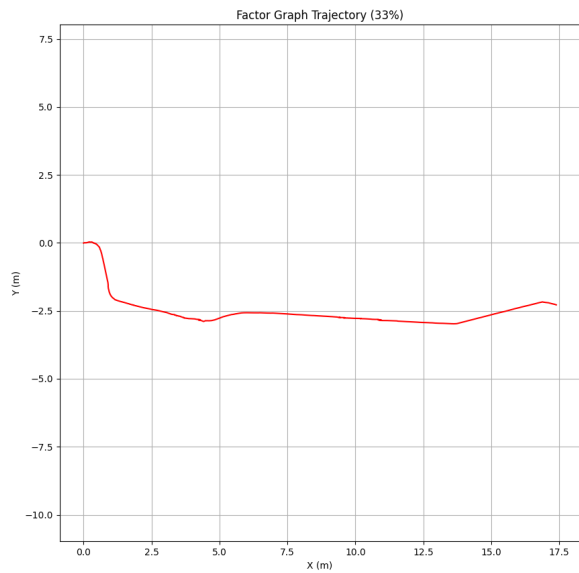


Fig. 26. Dataset 21: Factor-graph optimized trajectory (33%).

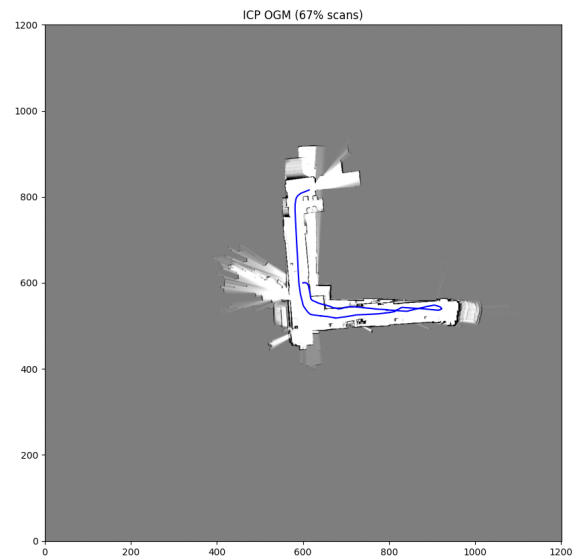


Fig. 28. Dataset 21: ICP refined OGM (67%).

2) 67%:

a) *OGM*: The following images are OGM:

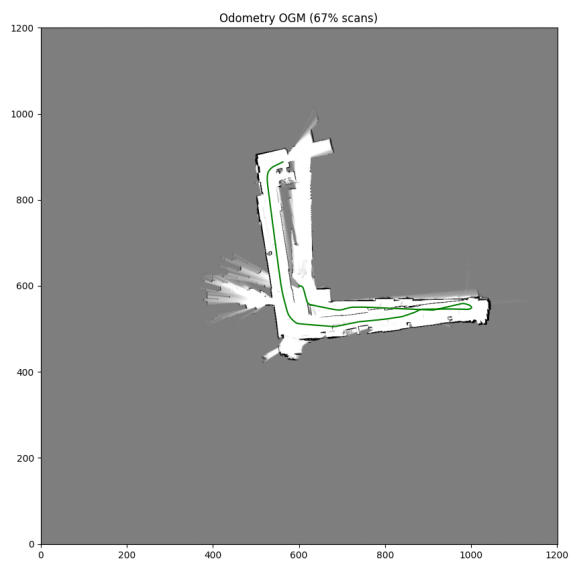


Fig. 27. Dataset 21: Odometry OGM (67%).

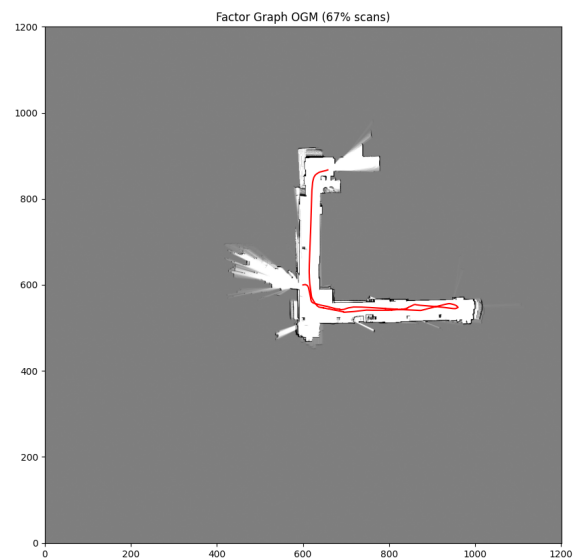


Fig. 29. Dataset 21: Factor-graph optimized OGM (67%).

b) *Trajectories*: The following images are the trajectories:

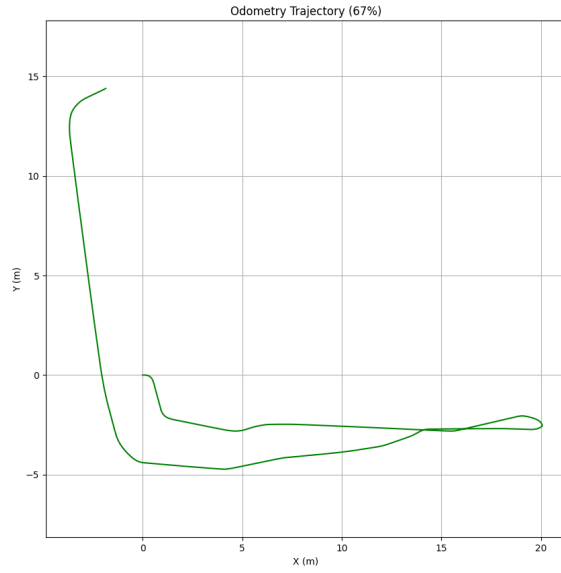


Fig. 30. Dataset 21: Odometry trajectory (67%).

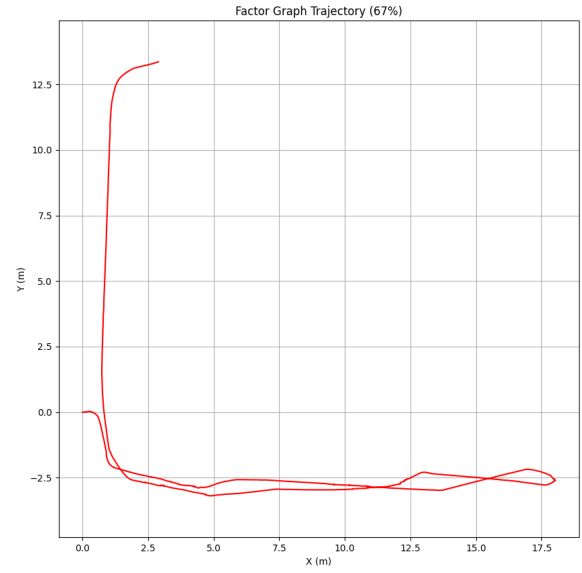


Fig. 32. Dataset 21: Factor-graph optimized trajectory (67%).

3) 100%:

a) OGM: The following images are OGM:

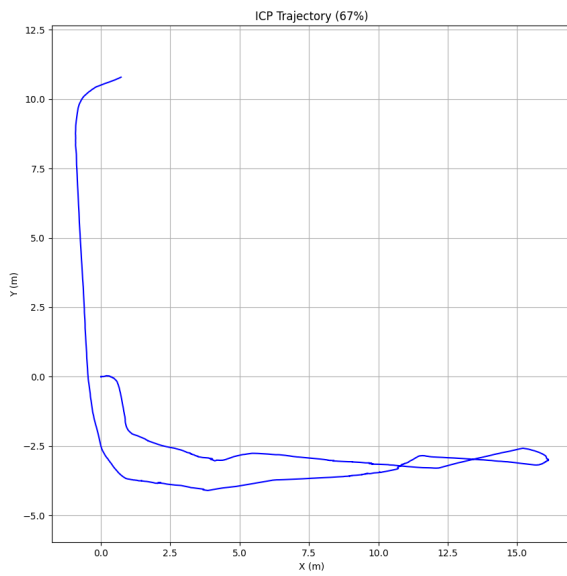


Fig. 31. Dataset 21: ICP refined trajectory (67%).

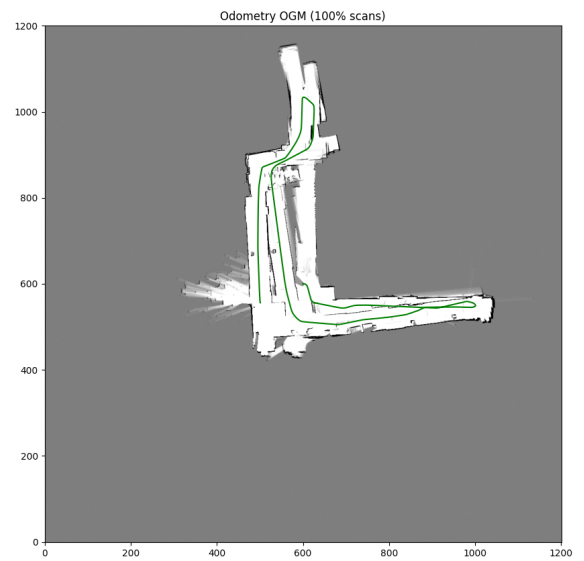


Fig. 33. Dataset 21: Odometry OGM (100%).

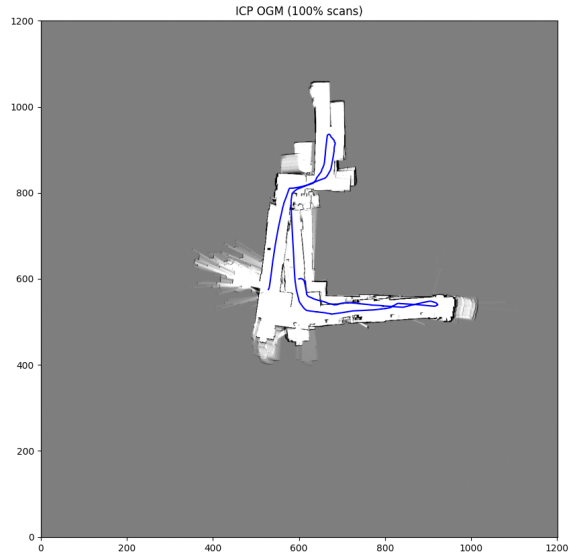


Fig. 34. Dataset 21: ICP refined OGM (100%).

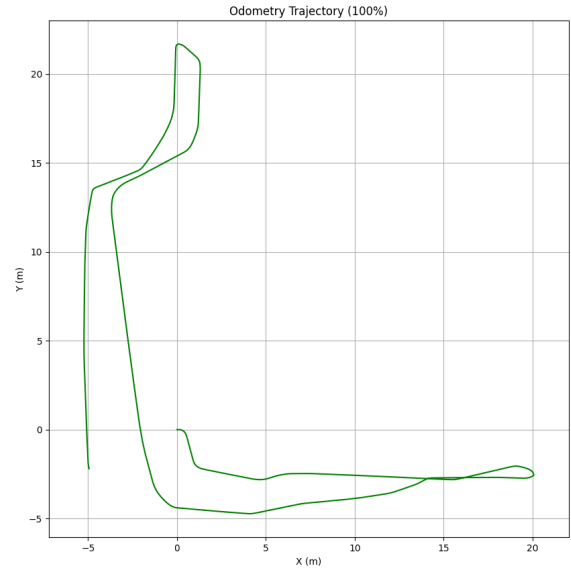


Fig. 36. Dataset 21: Odometry trajectory (100%).

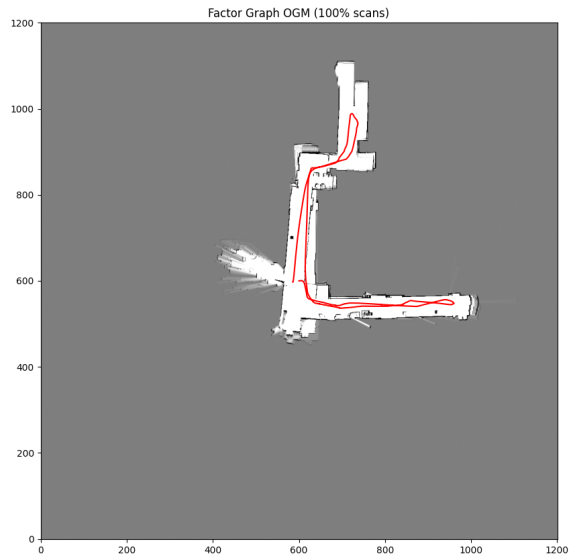


Fig. 35. Dataset 21: Factor-graph optimized OGM (100%).

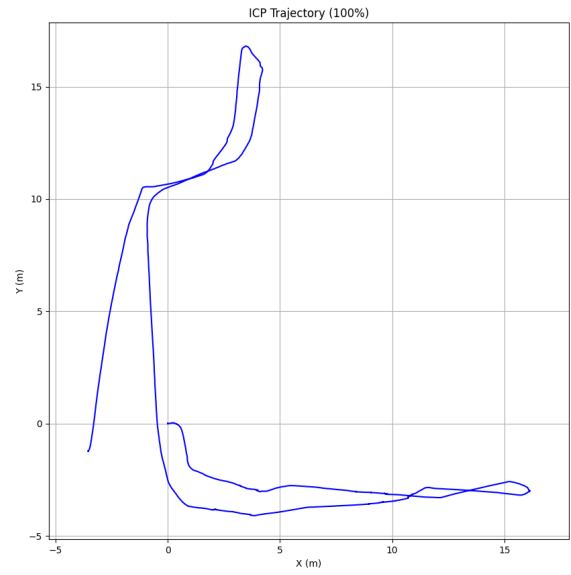


Fig. 37. Dataset 21: ICP refined trajectory (100%).

b) Trajectories: The following images are the trajectories:

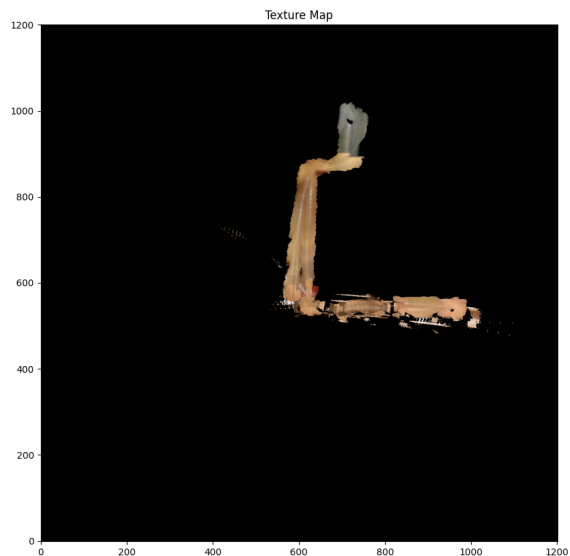
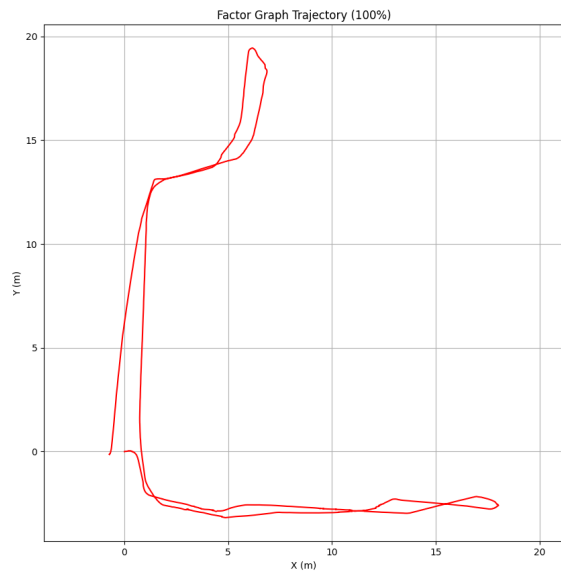


Fig. 38. Dataset 21: Factor-graph optimized trajectory (100%).

Fig. 40. Dataset 21: Final floor texture map.

C. Loop Closure Impact

When running the ICP scan-matching algorithm, it "tightened" the robot's trajectory and also altered the angle at which it moved, as can be seen in fig. 19 and fig. 39. However, this may have tightened the trajectory far too much. ICP also introduced ghosting for dataset 21, as can be seen in fig. 34. Introducing loop closure detection and factor graph optimization loosened the trajectory, causing it to appear closer to the original, as can be seen in 19 and fig. 39. It also fixed many of the issues that occurred with the robot's angle, making it much closer to the original odometry trajectory. Loop closure also tightened the robot's turns, which reduced ghosting and improved the overall OGM.

D. What Worked and What Did Not

Most of the methods worked during the development process. One key learning point was the necessity of constraints when working with loop detection and scan matching. This is because without constraints, erroneous scans would impact the trajectory by creating jagged edges, causing ghosting to appear on the OGM. Also, it is necessary to skip most of the points in loop detection, as it takes several hours running loop detection on every point. Although there may be a more intelligent way to both improve the runtime and achieve a better outcome, it was not implemented here.

Most of the issues during the development process occurred during ICP and loop-closure tuning. The ICP algorithm for scan-matching almost always produced a worse map than the original odometry, and during loop closure detection, my algorithm was finding nearly 400000 factors. This was fixed by tweaking the thresholds and adding skips to reduce the computation time and also improve the accuracy. At the very

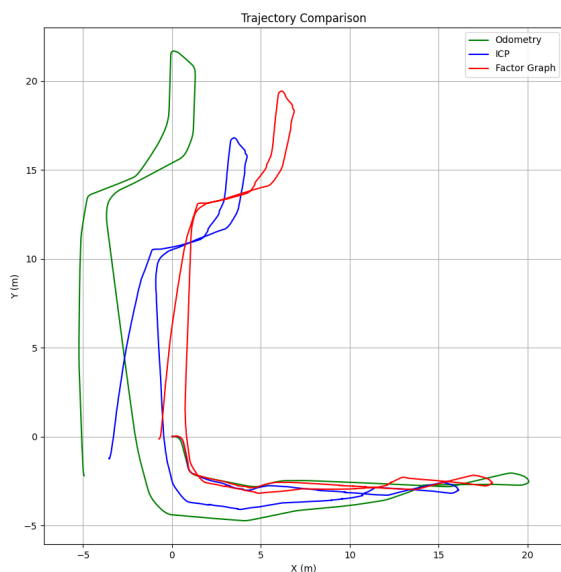


Fig. 39. Dataset 21: Trajectory comparison (odometry vs. ICP vs. factor graph).

end, there were only between 200-500 loop closure factors added.

REFERENCES

- [1] N. Atanasov, "ECE276A: Sensing & Estimation in Robotics Lecture 4: Motion and Observation Models." Accessed: Feb. 27, 2026. [Online]. Available: https://natanaso.github.io/ece276a/ref/ECE276A_4_MotionAndObservationModels.pdf
- [2] N. Atanasov, "ECE276A: Sensing & Estimation in Robotics Lecture 5: Factor Graph SLAM." Accessed: Feb. 27, 2026. [Online]. Available: https://natanaso.github.io/ece276a/ref/ECE276A_5_FactorGraphSLAM.pdf
- [3] N. Atanasov, "ECE276A: Sensing & Estimation in Robotics Lecture 6: Localization and Odometry." Accessed: Feb. 27, 2026. [Online]. Available: https://natanaso.github.io/ece276a/ref/ECE276A_6_LocalizationOdometry.pdf
- [4] F. Dellaert and M. Kaess, "Factor Graphs for Robot Perception," *Foundations and Trends in Robotics*, vol. 6, no. 1–2, pp. 1–139, 2017. [Online]. Available: <http://www.cs.cmu.edu/~kaess/pub/Dellaert17fnt.pdf> [4]